Projeto de Implementação de um Compilador para a Linguagem TPP

Análise Sintática (Trabalho -- 2a. parte)

Prof. Rogério Aparecido Gonçalves

> **Resumo:** *Este documento apresenta a especificação da 2a. parte do trabalho de implementação da disciplina. O objetivo nessa etapa é projetar e implementar a fase de _Análise Sintática* do compilador para a linguagem **TPP**. A Análise Sintática é a fase do compilador que recebe a lista de tokens e faz a validação das regras sintáticas da linguagem. O repositório *template* foi atualizado com o código inicial da implementação dessa fase._

Análise Sintática

A **Análise Sintática** determina a sintaxe ou estrutura de um programa. A sintaxe de uma *Linguagem de Programação (LP)* é normalmente dada pelas **regras gramaticais** de uma **Gramática Livre de Contexto (GLC)**. Assim como a estrutura léxica é determinada por expressões regulares, uma `GLC` utiliza convenções para nomes e operações muito similares às usadas por expressões regulares. A diferença é que as regras de uma gramática livre de contexto podem ser **recursivas**.

Existem duas categorias gerais de algoritmos para análise sintática: **Análise Sintática Ascendente** e **Análise Sintática Descendente**. O que determina como é feita a *análise sintática* e como *árvore sintática* correspondente é construída.

No processo de Análise Sintática o **Analisador Sintático** ou **Parser** é responsável por:

- determinar a estrutura sintática de um programa a partir das **marcas** (tokens) produzidos pelo processo de **varredura** (**scanner**, Analisador Léxico)
- construir, explícita ou implicitamente, uma árvore sintática que representa essa estrutura.
- ao final teremos uma Árvore Sintática construída com as derivações das **regras sintáticas**.
- O analisador sintático gera uma árvore sintática pela derivação das regras sintáticas da gramática e é o consumidor da sequência de marcas produzidas pelo *Scanner* ou processo de varredura.

Descrição da Gramática da Linguagem `TPP`

Uma especificação da gramática da Linguagem `TPP` está disponível neste link [1] e são listados no Código \ref{cod:tpp:grammar:rules}.

```
programa ::= lista_declaracoes
lista_declaracoes ::= lista_declaracoes declaracao NEWLINE
                    | declaracao NEWLINE
declaracao ::= declaracao_variaveis
             | inicializacao_variaveis
             | declaracao_funcao
declaracao_variaveis ::= tipo DOIS_PONTOS lista_variaveis
inicializacao_variaveis ::= lista_alvos ATRIBUICAO lista_expressoes
```

```
                            | ABRE_PARENTESE lista_alvos FECHA_PARENTESE ATRIBUICAO
lista_expressoes
lista_variaveis ::= lista_variaveis VIRGULA var
                  | var
var ::= ID
       | ID indice
indice ::= indice ABRE_COLCHETE expressao FECHA_COLCHETE
         | ABRE_COLCHETE expressao FECHA_COLCHETE
tipo ::= INTEIRO
        | FLUTUANTE
declaracao_funcao ::= tipo cabecalho
                     | tipos_retorno cabecalho
                     | cabecalho
tipos_retorno ::= ABRE_PARENTESE lista_tipos FECHA_PARENTESE
lista_tipos ::= lista_tipos VIRGULA tipo
               | tipo
cabecalho ::= ID ABRE_PARENTESE lista_parametros FECHA_PARENTESE NEWLINE bloco FIM
lista_parametros ::= lista_parametros VIRGULA parametro
                     | parametro
                     | vazio
parametro ::= tipo DOIS_PONTOS ID
             | parametro ABRE_COLCHETE FECHA_COLCHETE
bloco ::= corpo
         | corpo acao
corpo ::= corpo acao NEWLINE
         | vazio
acao ::= expressao
        | atribuicao
        | declaracao_variaveis
        | se
        | repita
        | leia
        | escreva
        | retorna
        | erro
se ::= SE expressao ENTAO bloco FIM
      | SE expressao ENTAO bloco SENAO bloco FIM
repita ::= REPITA bloco ATE expressao
atribuicao ::= lista_expressoes ATRIBUICAO lista_expressoes
              | lista_expressoes ATRIBUICAO atribuicao
lista_alvos ::= lista_alvos VIRGULA var
               | var
lista_expressoes ::= lista_expressoes VIRGULA expressao
                     | expressao
leia ::= LEIA ABRE_PARENTESE var FECHA_PARENTESE
escreva ::= ESCREVA ABRE_PARENTESE expressao FECHA_PARENTESE
retorna ::= RETORNA ABRE_PARENTESE lista_expressoes FECHA_PARENTESE
expressao ::= expressao_logica
expressao_logica ::= expressao_simples
                     | expressao_logica operador_logico expressao_simples
expressao_simples ::= expressao_aditiva
                      | expressao_simples operador_relacional expressao_aditiva
expressao_aditiva ::= expressao_multiplicativa
```

```
                      | expressao_aditiva operador_soma expressao_multiplicativa
expressao_multiplicativa ::= expressao_unaria
                           | expressao_multiplicativa operador_multiplicacao
expressao_unaria
expressao_unaria ::= fator
                   | operador_soma fator
                   | operador_negacao fator
operador_relacional ::= MENOR
                        | MAIOR
                        | IGUAL
                        | DIFERENTE
                        | MENOR_IGUAL
                        | MAIOR_IGUAL
operador_soma ::= MAIS
                  | MENOS
operador_logico ::= E
                  | OU
operador_negacao ::= NAO
operador_multiplicacao ::= VEZES
                           | DIVIDE
fator ::= ABRE_PARENTESE expressao FECHA_PARENTESE
          | var
          | chamada_funcao
          | numero
          | inicializacao_arranjo
          | tupla
tupla ::= ABRE_PARENTESE lista_expressoes VIRGULA expressao FECHA_PARENTESE
inicializacao_arranjo ::= ABRE_CHAVE lista_expressoes FECHA_CHAVE
numero ::= NUM_INTEIRO
           | NUM_PONTO_FLUTUANTE
           | NUM_NOTACAO_CIENTIFICA
chamada_funcao ::= ID ABRE_PARENTESE lista_argumentos FECHA_PARENTESE
lista_argumentos  ::= lista_argumentos VIRGULA expressao
                      | expressao
                      | vazio
```

Instruções Gerais

1. Faça *download* do arquivo do modelo de estrutura do trabalho e relatório disponível na página da disciplina no moodle. Descompacte e trabalhe nos arquivos e estrutura fornecida, pois será a mesma estrutura que deverá ser entregue ao final do projeto.
2. Siga a estrutura fornecida para desenvolver o trabalho.
3. O relatório deve ter a descrição do trabalho e da implementação e exemplos de entrada e saída gerada pelo Analisador Sintático.
4. Deverão ser entregues:
   a) O código fonte da implementação.
   b) Relatório em **pdf** que pode ser feito no formato do `LibreOffice` ou no `Latex`.
5. O projeto deve seguir a estrutura de diretórios e arquivos, disponível no formato. A estrutura do projeto é apresentada na Figura \ref{fig:formato:entrega}.

Implementação

A implementação da *Análise Sintática* deverá ser realizando utilizando o mesmo conjunto de ferramentas compatíveis utilizado na Análise Léxica. O código base para a implementação está disponível no link [Código Start da Sintática no Github Classroom](#) do Moodle.

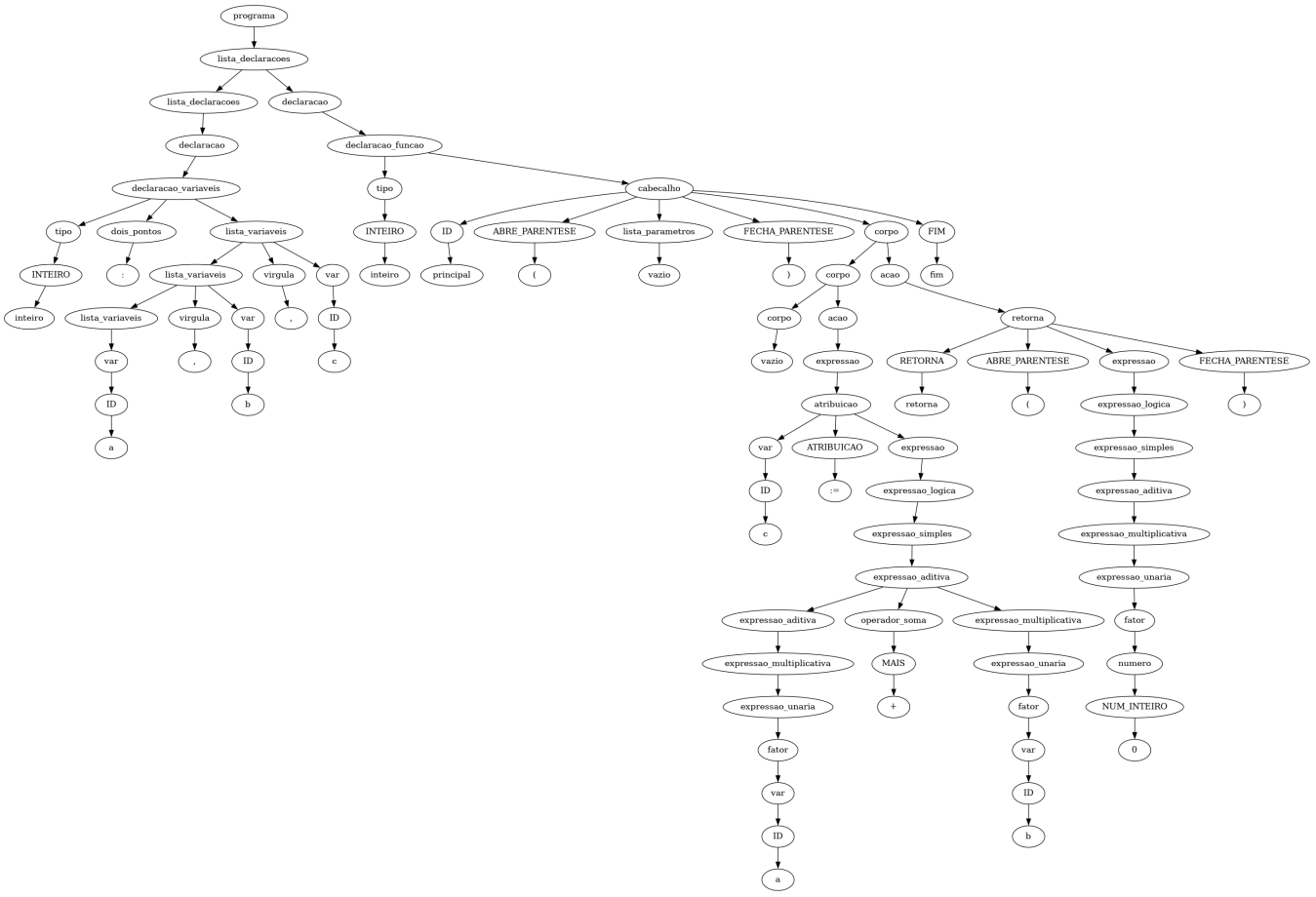
A análise sintática deverá ser implementada baseada em uma descrição formal da Gramática da Linguagem `TPP` (GLC - Gramática Livre de Contexto) presenta na documentação. A lista de símbolos analisados sintaticamente poderão ser obtidos da implementação da Análise Léxica.

A estrutura da **Árvore Sintática** será montada nessa fase. Para cada uma das regras sintáticas deverá ser observado a necessidade de adicionar um nó na árvore sintática correspondente à derivação da regra. Como resultado da implementação desta parte do trabalho, uma **árvore sintática** (AST) estruturando todo o código fonte do arquivo de entrada deverá ser montada. O Código \ref{cod:exemplo:tpp} apresenta um exemplo de um arquivo de entrada.

```
inteiro: a, b, c

inteiro principal()
  c := a + b
  retorna(0)
fim
```

A Árvore gerada com resultado do processo de Análise Sintática para o exemplo do Código \ref{cod:exemplo:tpp} é apresentada na Figura \ref{fig:ast:exemplo}.



Linguagens de programação para a implementação

Para a implementação do compilador, pode ser utilizado qualquer linguagem de programação [2]. É recomendado que seja utilizado uma linguagem que dê suporte à estruturas de dados de alto nível e preferencialmente que exista bibliotecas para a construção da varredura e gramática. Algumas LPs/ferramentas conhecidas são:

- `C/C++ - Flex/Bison` [@paxson1995flex], [@bison]
- `Python - PLY` (que possui ferramenta Léxica e Sintática) [@ply]
- `Java - JFlex/Jacc` [@jflex2020], [@jacc]

Obs.: A implementação de referência será apresentada na linguagem `Python`.

Para essa fase serão necessárias as instalações das bibliotecas do `Python`:

- `Anytree`: https://pypi.org/project/anytree/ para a implementação da árvore sintática.
- `Graphviz`: https://pypi.org/project/graphviz/ para a gerar a figura da árvore.

Testes

Alguns casos de testes estão disponíveis no moodle institucional junto com essa especificação no código inicial fornecido. O `Github Classroom` executará os testes automaticamente. Serão executados esses testes e outros testes que o professor julgar necessário durante a avaliação desta parte do trabalho.

- **Parser:** programa de exemplo `TPP` de **entrada** na linha de comando.
- **Saída:** Árvore Sintática e uma lista de erros que foram identificados.

No Código Start da Sintática no Github Classroom tem um diretório `tests` com os testes para a *Análise Sintática*.

É necessário terminar o desenvolvimento da parte de tratamento de erros. Está sendo disponibilizado uma classe de tratamento de erros que será utilizada em todas as fases do projeto, a `myerror.py` que carrega uma lista de erros composta por pares (chave:valor) formada por *códigos de erros* e as *mensagens de erros* correspondentes.

Na primeira fase provavelmente você alterou os arquivos relacionados à Análise Léxica, copie seus arquivos para atualizar as versões iniciais que estão nesse *Código de Start*.

Para que os testes possam ser automatizados foi disponibilizado um arquivo padrão que o `Github` utiliza para executar os testes a cada `commit` realizado. O sistema executa o `tppparser_test.py` que chama o `tppparser.py` da sua implementação com cada um dos arquivos de teste e compara se a saída produzida é a mesma saída esperada.

```
import pytest
import tppparser
import subprocess
import shlex
import os, fnmatch

test_cases = [("", "-k"), ("teste.c", "-k")]
```

```
for file in fnmatch.filter(os.listdir('tests/'), '*.tpp'):
    test_cases.append((file, "-k"))


@pytest.mark.parametrize("input_file, args", test_cases)
def test_execute(input_file, args):
    if(input_file != ''):
        path_file = 'tests/' + input_file
    else:
        path_file = ""

    cmd = "python tppparser.py {0} {1}".format(args, path_file)
    process = subprocess.Popen(shlex.split(cmd), stdout=subprocess.PIPE,
stderr=subprocess.PIPE)

    stdout, stderr = process.communicate()
    stdout, stderr

    path_file = 'tests/' + input_file
    output_file = open(path_file + ".syn.out", "r")

    #read whole file to a string
    expected_output = output_file.read()

    output_file.close()

    print("Generated output:")
    print(stdout)
    print("Expected output:")
    print(expected_output)

    assert stdout.decode("utf-8") == expected_output
```

A ideia é que em uma *execução normal* as mensagens de erros sejam recuperadas e impressas na saída padrão (terminal).

<!--

\newpage

Exemplo de saída para teste sem erros em *execução normal*:

[columns]

[column=0.5]

Arquivo: `teste-001.tpp`

```
inteiro: a,b
```

Execução:

```
$ python3 tppparser.py teste-001.tpp
```

[column=0.5]

Saída gerada:

```
INTEIRO
DOIS_PONTOS
ID
VIRGULA
ID
```

[/columns]

Exemplo de saída para teste com erro em *execução normal*:

[columns]

[column=0.5]

Arquivo: `teste-002.tpp`

```
inteiro: a,b
ç
```

Execução:

```
$ python3 tpplex.py teste-002.tpp
```

[column=0.5]

Saída gerada:

```
INTEIRO
DOIS_PONTOS
ID
VIRGULA
ID
Erro[02][01]: Caracter inválido: ç
```

[/columns]

-->

Já em uma *execução de testes automatizados* seja fornecido um parâmetro `-k` que imprima o código de erro apenas, possibilitando que o código apareça entre os *tokens* da lista de *tokens* gerada para o arquivo de entrada.

<!--

Exemplo de saída para teste sem erros em *execução de testes automatizados*:

[columns]

[column=0.5]

Arquivo: `teste-001.tpp`

```
inteiro: a,b
```

Execução:

```
$ python3 tpplex.py -k teste-001.tpp
```

[column=0.5]

Saída gerada:

```
INTEIRO
DOIS_PONTOS
ID
VIRGULA
ID
```

[/columns]

Exemplo de saída para teste com erros em *execução de testes automatizados*:

[columns]

[column=0.5]

Arquivo: `teste-002.tpp`

```
inteiro: a,b
ç
```

Execução:

```
$ python3 tpplex.py -k teste-001.tpp
```

[column=0.5]

Saída gerada:

```
INTEIRO
DOIS_PONTOS
ID
VIRGULA
ID
ERR-LEX-INV-CHAR
```

[/columns]
-->

O arquivo `ErrorMessages.properties` foi atualizado com a seção de erros e avisos para a *Sintática*:

```
[ParserErrors]
ERR-SYN-USE=Uso: python tppparser.py file.tpp
ERR-SYN-NOT-TPP=Não é um arquivo .tpp.
ERR-SYN-FILE-NOT-EXISTS=Arquivo .tpp não existe.
WAR-SYN-GEN-SYNTAX-TREE=Generating Syntax Tree Graph.
WAR-SYN-NOT-GEN-SYN-TREE=N\~ao foi poss\'ivel gerar a \'Arvore Sint\'atica.
ERR-SYN-IRRECUPERAVEL=Estado irrecuper\'avel.
WAR-SYN-ANA-SUCCESS=An\'alise Sint\'atica Executada com Sucesso. Gr\'afico da \'Arvore foi
gerado.
ERR-SYN-LISTA-ARGUMENTOS=Erro na lista de Argumentos.
ERR-SYN-INDICE=Erro na definicao do indice. Expressao ou indice.
ERR-SYN-EOF-INESPERADO=Erro inesperado.
WAR-SYN-OUTPUT-FILE=Output file:
```

Para a regra `lista_argumentos : lista_argumentos VIRGULA expressao` da `EBNF` que está implementada na função:

```
def p_lista_argumentos(p):
    """lista_argumentos : lista_argumentos VIRGULA expressao
                        | expressao
                        | vazio
        """
    pai = MyNode(name='lista_argumentos', type='LISTA_ARGUMENTOS')
    p[0] = pai

    if len(p) > 2:
        p[1].parent = pai

        filho2 = MyNode(name='VIRGULA', type='VIRGULA', parent=pai)
        filho_sym = MyNode(name=p[2], type='SIMBOLO', parent=filho2)
        p[2] = filho2

        p[3].parent = pai
    else:
        p[1].parent = pai
```

O tratamento de erro será feito na implementação de uma função com o mesmo nome adicionado do sufixo `_error` e com a definição de uma regra contendo a marcação da posição do *erro* com o identificador `error`:

```
def p_lista_argumentos_error(p):
    """lista_argumentos : error VIRGULA expressao
        """
    error_handler.newError('ERR-SYN-LISTA-ARGUMENTOS')
```

O padrão de nomes da chaves de erros deve considerar o nome da função que está sendo tratada. Do exemplo anterior, para a função `p_lista_argumentos_error` deverá ser emitida a chave de erro `ERR-SYN-LISTA-ARGUMENTOS`. Caso tenhamos mais de um erro a ser levantado na mesma função, deve ser tratado.

Na impressão feita pela classe `MyError`, que foi definida no arquivo `myerror.py`, a distinção entre erros e avisos deve ser feita.

Documentação

Durante toda a disciplina o aluno criará uma documentação formal da implementação do compilador para a linguagem. Sendo os relatório com conteúdo acumulativo, isto é, as fases subsequentes irão complementar o conteúdo existente das fases anteriores.

Para a fase de **Análise Sintática**, a documentação deve apresentar os seguintes itens:

- A descrição da Gramática no padrão BNF.
- Discussão sobre o formato na Análise Sintática realizado pela ferramenta (se é LL(1), LR(1), LALR(1), SLR(1), etc).
- Discussão sobre a implementação e utilização da ferramenta `Yacc`.
- Discussão sobre implementação da Árvore Sintática.

O relatório deve ser escrito no formato de artigo [3] disponível no link, faça uma cópia para seu `Overleaf`. O número mínimo em cada etapa é de 12 páginas.

Avaliação

A avaliação da Análise Sintática será realizada da seguinte forma:

Programa de exemplo `TPP` de **entrada** na linha de comando. **Saída** será a validação da análise sintática. Além disto uma **representação da árvore sintática abstrata** (ASA) deverá ser mostrada na saída do programa. A saída da análise sintática por meio de uma ASA é de extrema importância para a continuação da terceira parte do trabalho (Análise Semântica). Deverá ser possível verificar erros `shift-reduce` e `reduce-reduce` (debuggers deverão estar ativos para a realização da avaliação).

É importante que uma classe de análise de erros já esteja preparada. Os erros mais comuns em desenvolvimento de programas estão relacionados com a Análise Sintática.

- Serão avaliados, dentre outros critérios:
  a) Da implementação:
  + O funcionamento do programa.
  + O capricho e a organização na elaboração do projeto.
  + A corretude da implementação em relação ao que foi pedido no trabalho.
  + A colocação em prática dos conceitos que foram discutidos em sala de aula de forma correta.
  + A qualidade do projeto e da implementação (descrição e elaboração do projeto e o passo a passo da implementação).

  b) Do relatório:
  + O conteúdo e a forma que foi apresentado, se o formato é o mesmo solicitado.
  + Organização das ideias e do processo de tradução.
  + O capricho na elaboração e na formatação do texto, bem como o conteúdo do texto.
- Não serão avaliados os trabalhos:
  a) Que chegarem fora do prazo.
  b) Que não forem feitos nas ferramentas solicitadas.
  c) Que não estão no formato especificado.
  d) Que não foram compactados em um só arquivo.
  e) Que não tiverem identificação (nome e matrícula).

f) Que forem cópias de outros trabalhos ou materiais da internet.
g) Que não seguirem todas estas instruções.

- Não se esqueça que o trabalho vale **10,0** e contribui para o cálculo da nota final.

Entrega e Apresentação

O trabalho será **individual** e deverá ser entregue até o dia **14/05/2026** no moodle da disciplina em um pacote compactado. A estrutura do projeto com os arquivos do projeto (fonte e relatório) deve ser compactada (zipados) e o arquivo compactado deve ser enviado pelo moodle utilizando a opção de submissão **"Trabalho 2a. parte - Análise Sintática"**, o nome do arquivo compactado deve seguir o padrão de nomes do formato.

Deverá ser especificado na entrega o mecanismo de execução do *parser* para a realização da correção.

Obs.: Favor utilizar ZIP como forma de compactação. O Relatório deve ser entregue impresso, no horário da aula, para o professor.

\newpage

Leitura Recomendada {.allowframebreaks}

\normalsize

<!--

[alertblock]{Leitura Recomendada}

Capítulo 3: Gramáticas Livres de Contexto e Análise Sintática

Livro: `LOUDEN, Kenneth C. Compiladores: principios e praticas. Sao Paulo, SP: Thomson, c2004. xiv, 569 p. ISBN 8522104220.`

[/alertblock]

[exampleblock]{Leitura Recomendada}

Capítulo 3: Análise Sintática

Livro: `AHO, Alfred V. et al. Compiladores: principios, tecnicas e ferramentas. 2. ed. Sao Paulo, SP: Pearson Addison-Wesley, 2008. x, 634 p. ISBN 9788588639249.`

[/exampleblock]

[block]{Leitura Recomendada}

Capítulo 3: Analisadores Sintáticos (Parsers)

Livro: `COOPER, Keith D.; TORCZON, Linda. Construindo compiladores. Rio de Janeiro, RJ: Elsevier/Campus, 2014. xxiv, 656 p. ISBN 9788535255645. Disponível em: https://research.ebsco.com/linkprocessor/plink?id=debd0a1f-ba89-3ee8-b0a1-2c929486cc51. Acesso em: 12 mar. 2024.`

[/block]
-->

Word Cloud

Referências {.fragile .allowframebreaks}

1. *Gramática da Linguagem TPP:* https://docs.google.com/document/d/1e7_M-bD1RUbJAnyR8rZyJ35vKbYEN6KQG4l5L8FQ7_I/edit?usp=sharing ↩

2. Que tenha suporte às ferramentas de especificação do sistema de varredura e da gramática e que também tenha suporte ao LLVM. ↩

3. Formato para o relatório: https://www.overleaf.com/read/fcqmjkfgktgt ↩